

Programme PASS

Plateforme du Programme d'Accès aux Smartphones Subventionnés

Architecture cible sur Microsoft Azure, défendable juridiquement et opérable — de l'enrôlement du bénéficiaire à la supervision du programme.

Document d'architecture technique

Auteur : Chef de service Études et Architecture Data

Entité : Direction de la Transformation Digitale et de l'Innovation (DTDI)

Objet : Architecture de référence pour le passage en production

1. Contexte et principe directeur

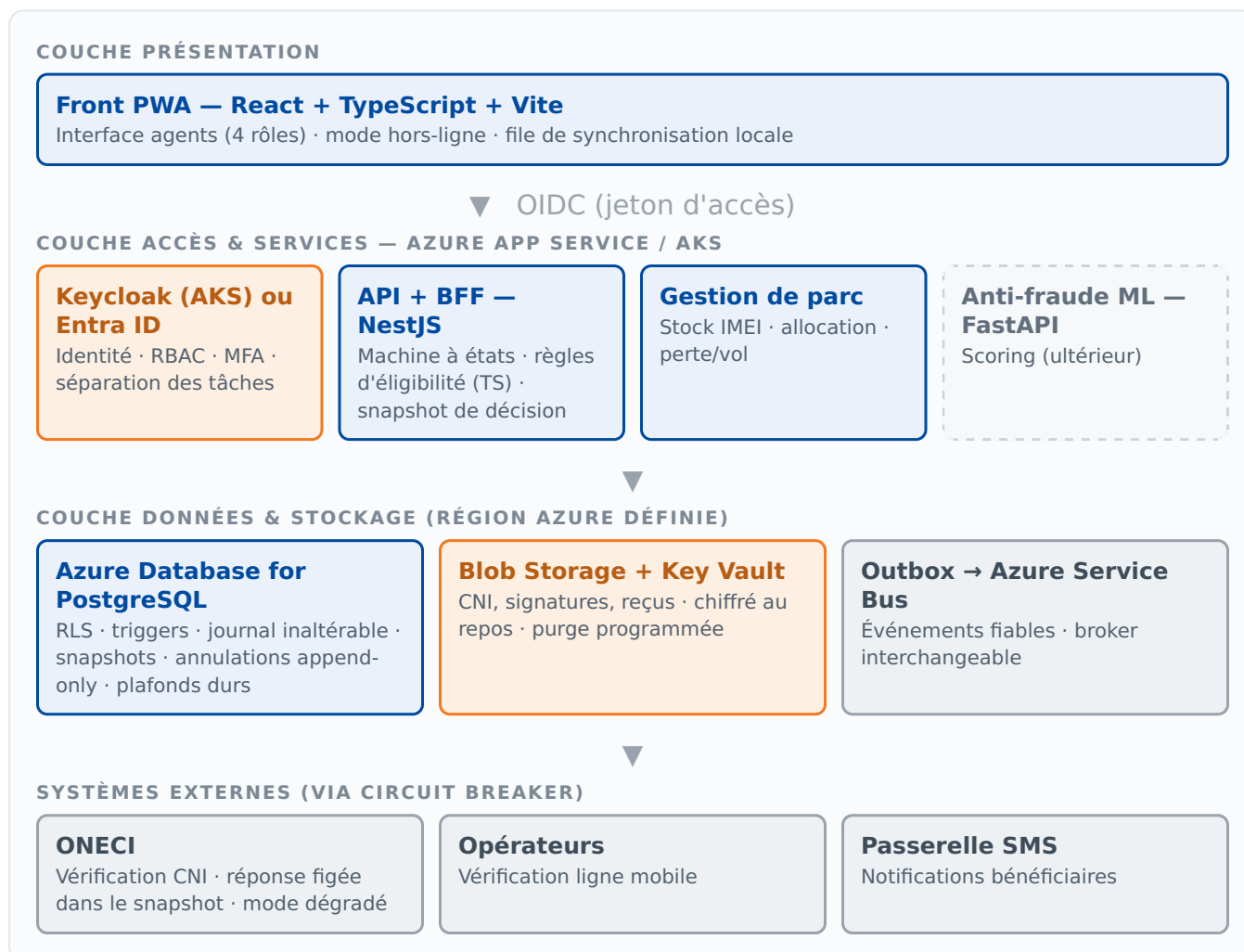
Le programme PASS met en œuvre le parcours **enrôlement assisté → instruction → décision d'éligibilité → remise du terminal → reçu bénéficiaire → supervision**. La présente architecture vise une mise en production sur **Microsoft Azure**, conforme au cadre légal ivoirien et durable dans son exploitation.

Principe directeur

Une attribution de subvention est un acte administratif **contestable par le citoyen**. Le cœur de valeur du système n'est pas « acheminer un terminal », mais **pouvoir prouver, des mois plus tard, pourquoi un citoyen a été retenu et un autre non — et pouvoir corriger proprement en cas d'erreur ou de fraude**. PASS est donc d'abord un **registre de décisions défendable**, auquel un workflow de distribution est attaché. L'auditabilité et la corrigibilité priment sur l'orchestration.

Deux règles d'architecture en découlent et structurent tout le document : les **garanties d'intégrité vivent en base de données** (seul endroit où une règle devient réellement inviolable), et **toute brique n'est introduite que lorsque son bénéfice est réel** (sobriété opérationnelle). Le choix d'Azure implique par ailleurs un traitement explicite de la **résidence et du transfert des données** (§4).

2. Vue d'ensemble de l'architecture cible



Trait plein = socle initial de production. Trait pointillé = extension ultérieure (montée en charge et détection de fraude par apprentissage).

3. Auditabilité et défendabilité juridique

3.1 — Le snapshot de décision immuable

Chaque décision d'éligibilité doit être **reproductible longtemps après** face à une réclamation. On stocke donc, attaché au dossier et immuable, un **snapshot auto-suffisant** :

- **Les données d'entrée** telles qu'utilisées au moment de la décision.
- **La version exacte du jeu de règles** qui a statué — non pas « les règles sont versionnées » en général, mais *quelle* version a jugé ce dossier.
- **La réponse ONECI figée** telle qu'elle était à l'instant de la décision : le statut CNI côté ONECI peut évoluer ensuite. Sans figer cette entrée externe, un replay produirait un résultat différent à règles identiques, et la décision ne serait plus prouvable.
- **Le résultat et l'horodatage.**

Exigence

Le snapshot doit permettre de **rejouer une décision sans dépendre d'un système vivant** — ni du magasin de règles courant, ni de l'état ONECI actuel. C'est la condition d'une décision défendable devant le citoyen comme devant le juge.

3.2 — « Irréversible » ne veut pas dire « incorrigible »

Le journal inaltérable et le caractère irréversible d'une décision servent l'audit ; mais une **fraude ou une erreur découverte après coup doit pouvoir être défaire**. Le pattern est comptable : jamais de mutation, toujours une **écriture compensatoire liée** (annulation en append-only).

- Le contrôle d'irréversibilité n'interdit pas « toute modification » mais « toute **mutation de l'écriture d'origine** » ; il autorise l'ajout d'un enregistrement d'annulation lié.
- **L'annulation est elle-même une décision** : elle exige le même snapshot auditable (qui a annulé, sur quelle base, avec quelle autorisation). À défaut, le vecteur de fraude se déplace simplement de « attribuer » vers « annuler puis réattribuer ».

Sans ce chemin de compensation tracé, le programme n'a **aucun moyen légal** de défaire une attribution frauduleuse.

4. Conformité, résidence et transfert des données

La loi n°2013-450 du 19 juin 2013 désigne l'ARTCI comme Autorité de protection : tout traitement de données à caractère personnel doit faire l'objet d'une **déclaration préalable ou d'une autorisation**. Pour un traitement de **données CNI par une entité publique, interconnecté aux fichiers ONECI**, c'est le régime le plus strict — **l'autorisation préalable** — qui s'applique.

Point spécifique au choix Azure

Les régions Azure les plus proches sont hors de Côte d'Ivoire (Afrique du Sud, Europe). Héberger des données CNI sur Azure constitue donc un **transfert de données à caractère personnel hors du territoire national**, soumis dans la loi 2013-450 à un régime propre : l'ARTCI délivre des **autorisations de transfert** nominatives. Ce n'est pas un obstacle en soi — l'usage du cloud public par des acteurs publics est courant — mais c'est une **formalité à obtenir explicitement**, en plus de l'autorisation de traitement.

Exigence légale	Traduction architecturale concrète
Autorisation de traitement ARTCI	Dossier à déposer en amont du développement ; à inscrire au planning dès le lancement.
Autorisation de transfert hors CI	Démarche distincte, liée au choix de la région Azure . À fixer tôt : la région conditionne latence, coût et dossier de transfert.
Résidence des données	Région Azure choisie et documentée ; chiffrement au repos (par défaut sur Blob Storage et Azure Database) et en transit ; clés gérées dans Key Vault.
Minimisation & finalité	Collecte et affichage limités au strict nécessaire <i>par rôle</i> (RLS + Keycloak/Entra ID). Le reçu n'expose ni IMEI ni géolocalisation.
Conservation & purge	Une horloge de conservation par catégorie de donnée et une purge automatique (règles de cycle de vie Blob Storage + purge en base) — un composant, pas une intention.
Droit d'accès du bénéficiaire	Journal d'accès orienté <i>personne concernée</i> , distinct de l'observabilité d'exploitation.
Analyse d'impact (DPIA)	À réaliser avant le développement — elle peut modifier des choix de schéma et de région.

Les modalités exactes (qualification du traitement, autorisation de transfert, délais, pièces) sont à confirmer avec le conseil juridique de l'ANSUT et l'ARTCI, qui mène en 2026 une campagne nationale d'évaluation de conformité à cette loi.

5. Rôle et intérêt de chaque composante

Front PWA — React + TypeScript + Vite

Application interne authentifiée pour les 4 rôles (Enrôlement, Instruction, Remise, Supervision). Aucun besoin de référencement ni de rendu serveur : React + Vite est le bon calibre. Hébergée en Azure App Service (ou conteneur sur AKS).

Pourquoi : déclinée en **PWA offline-first**, l'enrôlement se faisant sur le terrain, parfois hors couverture réseau. Une file de synchronisation locale enregistre les dossiers hors-ligne puis les rejoue à la reconnexion (voir §7 pour la résolution des conflits).

Identité — Keycloak (sur AKS) ou Microsoft Entra ID

Comptes agents, rôles (RBAC), MFA, anti-bruteforce, audit des connexions ; support de la **séparation des tâches** (§8).

Pourquoi ce choix ouvert : **Entra ID** est le service d'identité natif d'Azure, intégré et sans opération. **Keycloak sur AKS** reste pertinent si l'on veut une identité portable, indépendante du fournisseur cloud, et une **fédération future avec une identité gouvernementale** (OIDC/SAML) pleinement maîtrisée. À trancher selon le besoin de réversibilité.

API + BFF — NestJS

Cœur applicatif transactionnel : machine à états du parcours, évaluation des règles d'éligibilité (moteur léger en TypeScript, règles versionnées), production des **snapshots de décision**. Joue aussi le rôle de BFF. Hébergé en Azure App Service ou AKS.

Pourquoi (et pas une passerelle lourde) : pour 4 rôles internes derrière l'identité, NestJS couvre authentification, routage et agrégation. Azure API Management ne s'imposerait qu'en cas d'exposition multi-clients externe.

Azure Database for PostgreSQL — Données et intégrité

Service managé (Flexible Server) portant l'intégrité du programme : RLS, fonctions *SECURITY DEFINER*, triggers, journal d'audit inaltérable, snapshots de décision, annulations append-only, plafonds de quota.

Pourquoi : les garanties d'intégrité (unicité CNI, bénéfice non cumulable, décision irréversible, IMEI unique, plafond de campagne) restent en base — seul endroit où une règle devient réellement inviolable. Et comme cette logique est du **SQL standard**, elle est portable : aucun enfermement propriétaire, la **réversibilité** est préservée si l'on devait changer d'hébergeur.

Azure Blob Storage + Key Vault — Stockage documentaire

Objet-store managé pour les pièces sensibles (photos CNI, signatures, reçus), **chiffrées au repos par défaut**, clés gérées dans **Azure Key Vault**, et **règles de cycle de vie** pour la purge alignée sur la durée de conservation légale.

Pourquoi : ces fichiers n'ont pas leur place dans la base. Sur Azure, le stockage objet managé remplace un objet-store auto-hébergé — moins d'opération, chiffrement et cycle de vie natifs. Accès restreint par identités managées, jamais par clés en dur.

Gestion de parc — Stock IMEI

Sous-système de gestion des terminaux : réception, sérialisation, allocation par site, perte/vol, retour, réapprovisionnement (détaillé au §7).

Pourquoi : le lien **IMEI ↔ bénéficiaire est le contrôle anti-revente** — ce module est au cœur de l'intégrité du programme, pas un accessoire.

Pattern Outbox + Azure Service Bus — Intégration fiable

Le service métier écrit un message dans une table *outbox*, dans la même transaction que la décision ; un relais lit cette table et publie vers **Azure Service Bus**.

Pourquoi : zéro message perdu (cohérence garantie par la transaction) et broker interchangeable — le code métier ne connaît pas le bus, ce qui rend une future bascule (vers Event Hubs pour du flux à haut débit, par exemple) indolore côté applicatif.

Anti-fraude par apprentissage — FastAPI / Python (extension ultérieure)

Scoring de risque : doubles bénéfices, coordonnées partagées, réseaux de complicité difficiles à détecter par règles simples.

Pourquoi : écosystème Python naturel pour la data et le ML. Vient compléter — non remplacer — les garde-fous process de la phase initiale (§8).

6. Arbitrages structurants

6.1 — Orchestration : machine à états, pas moteur BPMN

Le parcours PASS est largement linéaire :



Quelques branches (« à instruire », rejet) et des passages de main humains — rien qui exige un moteur d'orchestration distribué. Une **machine à états dans NestJS** (statut + transitions autorisées) suffit, en restant dans l'écosystème TypeScript, sans introduire un 3^e environnement (JVM) à recruter et opérer.

Point d'arbitrage à poser explicitement

Le seul argument fort en faveur d'un moteur BPMN/DMN serait la **gouvernance** : si l'ANSUT exige que les décideurs *visualisent et éditent eux-mêmes* les règles. Dans ce cas, n'introduire d'abord que le moteur de décision (DMN), sans la plateforme d'orchestration complète. C'est une décision de gouvernance, à trancher en tant que telle.

6.2 — Bus de messages : Azure Service Bus d'abord

Le cœur transactionnel est synchrone et à faible débit. Un bus à haut débit en flux (type Event Hubs / Kafka) y serait une sur-ingénierie. Le pattern Outbox rendant le bus interchangeable, on retient **Azure Service Bus** pour le socle initial, et l'on réserve **Azure Event Hubs** à l'étape de détection de fraude en flux et de supervision analytique, le moment venu.

6.3 — Trois natures de règles, une source unique par valeur

Nature	Où elle vit — et comment
Invariants d'intégrité unicité CNI, décision irréversible, journal inaltérable, IMEI unique	En base (contraintes + triggers). Une garantie ne se délègue jamais.
Politique d'éligibilité configurable critères, catégories de bénéficiaires	Dans le moteur de règles TS (versionné). Change par campagne, pilotable par le métier.
Quota de campagne le plafond N	Valeur unique en configuration. Lue par les règles pour une décision <i>consultative</i> en amont, ET par la base pour un plafond dur atomique au commit (<i>SELECT ... FOR UPDATE</i> sur le compteur, pour arbitrer la course au dernier créneau).

Règle d'or : le trigger ne connaît pas le nombre, il le *lit*. Aucune valeur de quota n'est jamais recopiée — c'est ce qui interdit toute dérive entre les deux usages.

7. Mode hors-ligne et cycle de vie des terminaux

7.1 — Deux classes de conflit à la synchronisation

Principe fondateur : **un état local ne promet rien**. En mode déconnecté, l'application affiche « enregistré, en attente de synchronisation », jamais « confirmé ». Deux conflits distincts se résolvent au moment de la synchro, dans une même file de traitement :

Classe de conflit	Résolution
Collision d'unicité CNI deux agents, même bénéficiaire	Le serveur est seul arbitre : le premier écrit l'emporte, le second bascule en file de conflits pour traitement humain.
Quota épuisé à la synchro le plafond N est atteint avant le commit du dossier hors-ligne	Un dossier hors-ligne ne réserve pas de créneau . Conflit distinct de l'unicité, même file. « Enregistré hors-ligne » n'égale jamais « place garantie ».

Point structurant rassurant : **l'acte irréversible (la remise) se décide en ligne, en aval**. Le conflit remonte donc à l'instruction, *avant* tout acte irréversible ; le risque est réel mais borné à l'enrôlement, et rattrapable.

Décision d'équité à assumer

Pour la dernière place entre deux dossiers hors-ligne : arbitrer sur l'horodatage **local** (juste mais falsifiable) ou de **synchronisation** (infalsifiable mais pénalisant pour les zones mal couvertes) ? Pour un programme de **service universel**, pénaliser les zones enclavées serait un contresens. À trancher explicitement.

7.2 — Cycle de vie des terminaux (stock IMEI)

La remise lie un IMEI à un bénéficiaire, mais la gestion de parc est un sous-système à part entière :



Fonctions minimales : réception et sérialisation, allocation par site, réapprovisionnement, traitement perte/vol/retour, inventaire. **Décision à poser** : construire ce module dans PASS, ou l'intégrer à un outil de gestion d'actifs existant ? Au minimum, désigner où vit l'inventaire de référence.

8. Lutte contre la fraude

Le vecteur de fraude principal d'un tel programme est **l'agent lui-même**. Des garde-fous *process*, peu coûteux, sont mis en place dès le socle initial, sans attendre l'apprentissage automatique :

Contrôle	Mise en œuvre
Séparation des tâches	L'agent qui enrôle n'est pas celui qui remet. Du RBAC pur — porté par l'identité (Keycloak/Entra ID).
Double contrôle (4 yeux) à la remise	La remise, acte irréversible, exige une validation par un second rôle.
Supervision aléatoire	Re-vérification d'un échantillon de dossiers par un superviseur.
Seuils de vélocité	Alerte simple (ex. nombre de remises par agent et par jour) — de la requête SQL, pas du ML.

L'anti-fraude par apprentissage (§5) complète ensuite ces règles en détectant les *patterns* — réseaux de complicité, coordonnées partagées — invisibles aux contrôles simples.

9. Résilience et exploitation

9.1 — Résilience de la dépendance ONECI

La vérification CNI auprès de l'ONECI est le principal point de dépendance externe. Le mode dégradé est un chemin nominal, pas une exception : **circuit breaker** sur l'appel, **file** « à instruire » pour les dossiers non vérifiables en temps réel, **cache** des vérifications récentes, **SLA de reprise** défini. La réponse ONECI est par ailleurs figée dans le snapshot de décision (§3.1).

9.2 — Exploitation sur Azure

Domaine	Exigence minimale
Sauvegardes	Sauvegardes automatiques d'Azure Database for PostgreSQL et redondance de Blob Storage ; restaurations testées régulièrement (une sauvegarde non testée n'existe pas).
RPO / RTO / PRA	Objectifs de perte et de reprise définis ; stratégie de redondance de région le cas échéant ; plan de reprise documenté et exercé.
Observabilité	Azure Monitor / Log Analytics : logs centralisés, métriques, alerting ; traçabilité des accès aux données CNI .
Secrets & identités	Aucun secret en dur : Azure Key Vault + identités managées entre services. Aucun fichier d'environnement versionné.
CI/CD & environnements	Pipelines automatisés (Azure DevOps) ; environnements dev / recette / production séparés ; migrations de base versionnées et rejouables ; infrastructure décrite en code (Bicep / Terraform).

10. Prérequis et questions ouvertes

Cette architecture est une cible cohérente. Sa validation définitive et son dimensionnement supposent de lever, en amont, plusieurs inconnues qui relèvent du projet et non de la technique :

Sujet	À confirmer avant industrialisation
Volumétrie & cible	Nombre de bénéficiaires visés, nombre de sites, profil du pic de campagne. Détermine le dimensionnement des services Azure et le coût.
Accès ONECI	Existence d'une interface ONECI accessible à l'ANSUT, cadre juridique d'interconnexion, SLA réel. Risque projet majeur : tout le §9.1 en dépend.
Région Azure & transfert	Région retenue (latence, coût, résidence) et obtention de l' autorisation de transfert ARTCI correspondante (§4).
Souscription & landing zone	Souscription Azure de l'ANSUT, zone d'atterrissage, réseau, politique de gouvernance cloud déjà en place à réutiliser.
Identité : Keycloak ou Entra ID	Choix selon le besoin de réversibilité et de fédération gouvernementale (§5).
Équipe & compétences	Capacité à opérer la stack (NestJS, PostgreSQL avancé, identité, Blob/Key Vault, Service Bus, PWA, Azure), en interne ou via prestataire.
Terrain	Profil des agents, matériel, littératie numérique — déterminants pour l'ergonomie et la fiabilité du mode hors-ligne.

11. Synthèse

PASS est un **registre de décisions défendable** avant d'être un workflow de distribution. L'architecture retient : une décision **reproductible et figée** (snapshot auto-suffisant), **corrigible** par compensation tracée, **conforme** à la loi n°2013-450 (traitement et transfert autorisés), une intégrité **gardée en base** et portable (réversibilité préservée), un stockage CNI **chiffré et purgé**, un parc IMEI **maîtrisé**, et une **lutte anti-fraude dès le socle initial**. Les choix de mise en œuvre privilégient la **sobriété** (machine à états plutôt que moteur BPMN, Service Bus plutôt que bus à haut débit) et s'appuient sur les **services managés Azure** pour réduire la charge d'exploitation.

En une phrase

Une architecture Azure sobre et opérable, à la résidence et au transfert de données explicitement encadrés, dont la décision d'attribution est prouvable des mois plus tard et corrigible proprement — la condition d'un programme d'État défendable devant le citoyen.